Product Requirements Document

World Food Waste Analytics Chatbot

Learning & Portfolio Project
Version 0.5 — Phases 1–5 Complete, Phase 6 In Progress

---

1. Project Overview

Field	Detail
Project Name	World Food Waste Analytics Chatbot
Type	Learning & Portfolio Project
Owner	Data Engineer
Purpose	Learn Claude/Anthropic API in practice; build a portfolio-ready project; gain MCP server experience applicable to workplace
Domain	World Food Waste — public datasets (FAO, WDI, PIK, EDGAR), meticulously cleaned and processed as part of original thesis research
Stack	Python · SQLite · Streamlit · Anthropic Python SDK
Dev Environment	VS Code + Claude Code Extension
Repo Name	food-waste-analytics-chatbot
Repo Visibility	Public — live demo deployed on Streamlit Community Cloud
Live Demo	https://adilsahab16-food-waste-analytics-chatbot.streamlit.app
Version	v0.5 — Phases 1–5 Complete, Phase 6 In Progress

2. Goals & Success Criteria

Primary Goals

- Learn how Claude and the Anthropic API work — tool calling, agentic loop, MCP
- Build a working analytics chatbot answering natural language questions over food waste data
- Produce a portfolio-ready project: clean architecture, documented decisions, clear README
- Gain MCP server experience applicable to workplace use case

Out of Scope (For Now)

- Machine learning or predictive analytics — deferred to future phase

- Multi-user auth or cloud database — SQLite for local dev, sample dataset for demo

3. Data

Sources

All source datasets are publicly available. The cleaning, processing, and structuring work was carried out as part of original thesis research.

Source	Data Provided	Fact / Dimension Table
FAO	Food loss/waste percentages by country, commodity, supply stage	`fact_food_loss`
FAO	Food system emissions by country, item, gas type	`fact_food_system_emissions`
PIK	Total economy-wide GHG emissions by country, sector, gas	`fact_total_emissions_pik`
EDGAR	Food system emission shares (fractions) by country	`fact_food_emission_shares_edgar`
FAO	Country → region mapping, income classification	`dim_region`
FAO	Commodity → group → basket hierarchy	`dim_commodity`
WDI	Annual population by country	`dim_population`
WDI	Annual GDP by country	`dim_gdp`

Data Handling

- Full cleaned files are gitignored and never committed to the repository
- A curated sample dataset (`data/sample/`) is committed and powers the live Streamlit demo
- `DATA_MODE` environment variable controls whether the app loads full data (local dev) or sample data (demo/deployment)
- Sample covers 4 regions (Europe, SSA, S&SE Asia, LA) and 2005–2022 — sufficient for all 4 analytical groups
- Repo is public; full cleaned files remain local only

4. Project Phases & Status

Phase	Name	Status
1	Problem Definition & Use Case	✅ **Complete**
2	Data & Schema Design	✅ **Complete**
3	Question → SQL Mapping	✅ **Complete**
4	Tool / Function Design	✅ **Complete**
5	Agent Behavior & Streamlit UI	✅ **Complete**
6	MCP Server	🔄 **In Progress**

5. Phase 5 — Session Plan

All 6 sessions complete. Each session was completed, tested, committed to GitHub, and followed by a `/clear`.

Session	Goal	Key Output	Status
1	Repo Scaffold	Folder structure, requirements.txt, .gitignore, README skeleton, initial commit	✅ **Complete**
2	Data Loading	`src/load_data.py` — reads source files, applies all ingestion transformations, loads SQLite	✅ **Complete**
3	Tool Functions	`src/tools.py` — all 5 tool functions executing parameterised SQL against SQLite	✅ **Complete**
4	Agentic Loop	`src/agent.py` — Claude + tool calling loop via Anthropic SDK	✅ **Complete**
5	Streamlit UI	`src/app.py` — chat interface wired to agent with conversation starters, sidebar, markdown rendering, spinner	✅ **Complete**
6	Sample Data & Deploy	`data/sample/` files, `src/create_sample.py`, `DATA_MODE` config, updated README, Streamlit Community Cloud deployment	✅ **Complete**

Development Rhythm

- Attach `CLAUDE_CONTEXT.md` at the start of every session
- Tell Claude Code which session you are on and which files already exist
- Complete the task → test it → commit to GitHub → run `/clear` → start next session
- Never run `/clear` mid-task — always complete and commit first
- If a session runs more than 30–40 minutes without a commit, run `/clear` before continuing
- Claude Code will remind you to commit and clear at the end of each completed task

6. Phase 6 — MCP Server Session Plan

Expose the same 5 tools over the MCP (Model Context Protocol) so any MCP-compatible client — including Claude Desktop and VS Code extensions — can query the food waste database without the Streamlit UI. MCP architecture is directly applicable to a workplace integration.

Session	Goal	Key Output	Status
1	MCP Server Scaffold	`mcp/server.py` — FastMCP server exposing all 5 tools over MCP protocol	⏳ **Planned**
2	MCP Client Test	Verify tools callable from Claude Desktop or VS Code MCP client; end-to-end test	⏳ **Planned**
3	README + Final Commit	Update README with MCP server setup instructions; final public commit	⏳ **Planned**

Target Architecture (Post Phase 6)

```
Claude (Desktop / VS Code / any MCP client)
  │
  ▼
MCP Server  (mcp/server.py — FastMCP)
  │
  ├──► query_food_loss(...)                  ──► SQLite (src/tools.py)
  ├──► query_food_system_emissions(...)       ──► SQLite
  ├──► query_population_gdp(...)             ──► SQLite
  ├──► query_total_emissions_by_sector(...)   ──► SQLite
  └──► query_total_ghg_with_food_share(...)   ──► SQLite
```

7. Analytical Questions (Phase 1 Output)

Group 1 — Yearly Trends: High Income vs Low Income Regions

ID	Question	Notes
Q1.1	Yearly trend in food loss/waste % — high income vs low income	
Q1.2	Yearly trend in food system emissions — high income vs low income	
Q1.3	Population & GDP trends by income group	Contextual enrichment only — not standalone

Group 2 — Food Loss Breakdown

ID	Question	Notes
Q2.1	Food loss by supply chain stage — broken down by region	Regional breakdown, not global aggregate
Q2.2	Food loss of each region, broken down by commodity basket	

Group 3 — Food System Emissions Breakdown

ID	Question	Notes
Q3.1	Food system emissions by supply chain stage — by region	8 specific FAO item values
Q3.2	Food system emissions by region, by gas type	CH4, N2O, CO2eq AR5
Q3.3	Crops vs Livestock emissions — by region	Added during Phase 3

Group 4 — Total GHG Emissions

ID	Question	Notes
Q4.1	Total GHG by region with food system emission share	PIK + EDGAR combined in single tool
Q4.2	Total GHG by region by sector	PIK only. KYOTOGHG excluded

8. Data Model (Phase 2 Output)

Join Key Paths

- `m49_code` — links `fact_food_loss` and `fact_food_system_emissions` to `dim_region`
- `country_code` — links `fact_total_emissions_pik`, `fact_food_emission_shares_edgar`, `dim_population`, `dim_gdp` to `dim_region`

Dimension Tables

dim_region | Source: country_income_region_mapping

Column	Type	Notes
region	string	NAOCE, NAWACA, SSEA, SSA + others
country	string	Country name
country_code	string	Join key for WDI, PIK, EDGAR data
m49_code	integer	UN M49 code — join key for FAO data
dev_country	string	D = Developing, I = Industrialised
income	string	High or Low

dim_commodity | Source: commodity tagging

Column	Type	Notes
cpc_code	integer	Join key to fact_food_loss
commodity	string	Individual commodity
commodity_group	string	Intermediate grouping
basket	string	Top-level grouping — 6 baskets

dim_population | Source: world_bank_population_data

Column	Type	Notes
country	string	Country name
country_code	string	Join key to dim_region
year	integer	Unpivoted from wide format. 1960–2022
population	integer	Annual headcount

dim_gdp | Source: world_bank_gdp_data

Column	Type	Notes
country	string	Country name
country_code	string	Join key to dim_region
year	integer	Unpivoted from wide format. 1960–2022. 2023 dropped at ingestion
gdp	integer	Total GDP in current USD

Fact Tables

fact_food_loss | Source: World_Food Loss and Waste Database_FAO_1966-2022

Column	Type	Notes
m49_code	integer	Join key → dim_region
country	string	Country name
year	integer	From 1966
loss_percentage	float	% food wasted vs production. Fractions not quantities
food_supply_stage	string	e.g. Storage, Harvest, Farm, Processing, Transport, Households
cpc_code	integer	Join key → dim_commodity
commodity	string	Commodity name
source_name	string	Fixed: 'FAO'

Already tall format — add source_name column only

fact_food_system_emissions | Source: Emissions_Totals_E_All_Data_NOFLAG

Column	Type	Notes
m49_code	integer	Join key → dim_region. Excel apostrophe prefix stripped at ingestion
area	string	Country name
year	integer	Y-prefix stripped. Y2030/Y2050 dropped
emissions	float	Emissions in kt
item	string	Supply chain stage or crops/livestock category
element	string	CH4, N2O, or CO2eq AR5
source_name	string	Fixed: 'FAO'

Transformations: unpivot Y-prefixed years; keep FAO TIER 1 only; keep element IN (CH4, N2O, CO2eq AR5); drop projection years; strip Excel apostrophe prefix from m49_code

Supply chain items (Q3.1): Food Household Consumption, Waste, Pre- and Post- Production, Food Packaging, Agrifood Systems Waste Disposal, Food Retail, Food Transport, Food Processing

Crops/Livestock items (Q3.3): Emissions from crops, Emissions from livestock

fact_total_emissions_pik | Source: CW_HistoricalEmissions_PIK

Column	Type	Notes
country_code	string	Join key → dim_region
year	integer	Trimmed to 1966+ at ingestion
sector	string	e.g. Agriculture, Energy, Industrial Processes
gas	string	e.g. CH4, N2O, CO2. KYOTOGHG excluded
emissions	float	Emissions in kt
source_name	string	Fixed: 'PIK'

Transformations: unpivot years (1850–2021); trim to >= 1966; exclude KYOTOGHG; drop Source column

fact_food_emission_shares_edgar | Source: EDGAR-FOOD_EMISSIONS_SHARES

Column	Type	Notes
country_code	string	Renamed from Country_code_A3. Join key → dim_region
country	string	Renamed from Name
year	integer	Unpivoted. Range: 1970–2018
substance	string	Renamed from Substance. Averaged across at query time
share	float	Food emission share as fraction (0–1)
source_name	string	Fixed: 'EDGAR'

Transformations: unpivot year columns; rename columns; add source_name

9. Question → SQL Mapping (Phase 3 Output)

ID	Tables	Join Key	Group By	Measure	Key Filters
Q1.1	fact_food_loss, dim_region	m49_code	year, income	AVG(loss_%)	None
Q1.2	fact_food_system_emissions, dim_region	m49_code	year, income	SUM(emissions)	CO2eq AR5
Q1.3	dim_population, dim_gdp, dim_region	country_code	year, income	SUM(pop), AVG(gdp)	Two subqueries
Q2.1	fact_food_loss, dim_region	m49_code	region, supply_stage	AVG(loss_%)	None
Q2.2	fact_food_loss, dim_region, dim_commodity	m49_code + cpc_code	region, basket	AVG(loss_%)	None
Q3.1	fact_food_system_emissions, dim_region	m49_code	region, item	SUM(emissions)	CO2eq AR5 + 8 items
Q3.2	fact_food_system_emissions, dim_region	m49_code	region, element	SUM(emissions)	CH4, N2O, CO2eq AR5
Q3.3	fact_food_system_emissions, dim_region	m49_code	region, item	SUM(emissions)	CO2eq AR5 + crops/livestock
Q4.1	fact_total_emissions_pik, fact_food_emission_shares_edgar, dim_region	country_code	region	SUM(kt) + AVG(share)	Excl. KYOTOGHG. CTEs to avoid row fan-out
Q4.2	fact_total_emissions_pik, dim_region	country_code	region, sector	SUM(emissions)	Excl. KYOTOGHG

10. Tool Specifications (Phase 4 Output)

5 tools. All include `limit` parameter (default: 100). `group_by` is a dynamic list — Claude selects from the constrained valid set.

Tool 1: query_food_loss

Description: "Use this tool to query food loss and waste percentage data. Call this when the user asks about food loss or waste percentages across any combination of: time trends, regions, income groups, supply chain stages, commodity groups, or commodity baskets. This tool does NOT cover emissions."

Parameter	Type	Required	Notes
`group_by`	list[string]	Yes	Any of: year, income, region, food_supply_stage, basket
`filters`	object	No	Optional: region, income (High/Low), year_from, year_to, basket, food_supply_stage
`limit`	int	No	Default: 100

Returns: {group_by columns, avg_loss_percentage} | Serves: Q1.1, Q2.1, Q2.2

Tool 2: query_food_system_emissions

Description: "Use this tool to query food system emissions from the FAO. Call this when the user asks about agrifood system emissions, emissions by supply chain stage, emissions by gas type, crop emissions, or livestock emissions. For total economy-wide emissions use the total emissions tool."

Parameter	Type	Required	Notes
`group_by`	list[string]	Yes	Any of: year, income, region, item, element
`item_filter`	string	No	'supply_chain' (8 stages), 'crops_livestock', 'all'. Default: 'all'
`element_filter`	string	No	'co2eq' (AR5 only), 'by_gas' (CH4, N2O, CO2eq), 'all'. Default: 'co2eq'
`filters`	object	No	Optional: region, income, year_from, year_to
`limit`	int	No	Default: 100

Returns: {group_by columns, total_emissions_kt} | Serves: Q1.2, Q3.1, Q3.2, Q3.3

Tool 3: query_population_gdp

Description: "Use this tool to retrieve population and GDP per capita data as contextual support. Call this when the user asks about demographic or economic trends alongside food loss or emissions. Do not use as a primary analytical tool — it provides supporting context only."

Parameter	Type	Required	Notes
`group_by`	list[string]	Yes	Any of: year, income, region
`filters`	object	No	Optional: region, income, year_from, year_to
`limit`	int	No	Default: 100

Returns: {group_by columns, total_population, avg_total_gdp_usd, avg_gdp_per_capita} | Serves: Q1.3

Tool 4: query_total_emissions_by_sector

Description: "Use this tool to query total economy-wide GHG emissions by sector, sourced from PIK. Call this when the user asks about total emissions by sector (Agriculture, Energy, Industry etc.) across regions. Excludes KYOTOGHG. For food system emission shares, use query_total_ghg_with_food_share."

Parameter	Type	Required	Notes
`group_by`	list[string]	Yes	Any of: year, income, region, sector, gas
`filters`	object	No	Optional: region, income, sector, gas, year_from, year_to
`limit`	int	No	Default: 100

Returns: {group_by columns, total_emissions_kt} | Serves: Q4.2

Tool 5: query_total_ghg_with_food_share

Description: "Use this tool when the user asks about total GHG emissions by region combined with the food system's share. Joins PIK totals with EDGAR food emission shares and returns both values per region. Use for questions comparing overall emissions magnitude against food system contribution."

Parameter	Type	Required	Notes
`group_by`	list[string]	Yes	Any of: region, income, year
`filters`	object	No	Optional: region, income, year_from, year_to
`limit`	int	No	Default: 100

Returns: {group_by columns, total_emissions_kt, avg_food_share_pct} | Serves: Q4.1

Implementation note: uses CTEs to pre-aggregate PIK and EDGAR separately before joining — avoids row fan-out (PIK has ~20 rows/country/year, EDGAR has ~9)

11. Architecture

Post Phase 5 — Deployed

```
User
  │
  ▼
Streamlit UI  (src/app.py)
  │
  ▼
Anthropic Python SDK  (src/agent.py — agentic loop)
Claude decides: call tool OR answer directly
  │
  ├──► query_food_loss(...)                  ──► SQLite (src/tools.py)
  ├──► query_food_system_emissions(...)       ──► SQLite
  ├──► query_population_gdp(...)             ──► SQLite
  ├──► query_total_emissions_by_sector(...)   ──► SQLite
  └──► query_total_ghg_with_food_share(...)   ──► SQLite
                                                    │
                                  data/raw/        (full — local only, gitignored)
                                  data/sample/     (demo — committed to repo)
                                  controlled by DATA_MODE env variable
```

Post Phase 6 — With MCP (Planned)

```
Claude (Desktop / VS Code / any MCP client)
  |
  ▼
MCP Server  (mcp/server.py)  ──▶  same 5 tools over MCP protocol  ──▶  SQLite
```

Deployment

- Local development: full data via `data/raw/` , `DATA_MODE=full`
- Live demo: Streamlit Community Cloud, `DATA_MODE=sample` set as Streamlit secret, reads from `data/sample/`
- Live demo URL: `https://adilsahab16-food-waste-analytics-chatbot.streamlit.app`
- No cloud database required — SQLite with sample data is sufficient for the demo

12. Repository Structure

```
food-waste-analytics-chatbot/
├── CLAUDE_CONTEXT.md          ← Attach to every Claude Code session
├── README.md                  ← Portfolio-facing: architecture, setup, data sources
├── .gitignore                 ← Covers data/raw/, db/, .env
├── requirements.txt
├── data/
│   ├── raw/                  ← Full cleaned files (gitignored — never committed)
│   └── sample/               ← Curated demo subset (committed — powers live demo)
├── db/
│   └── food_waste.db         ← SQLite database (gitignored — generated locally)
├── src/
│   ├── load_data.py          ← Reads files, applies transformations, loads SQLite
│   ├── tools.py              ← 5 parameterised tool functions
│   ├── agent.py              ← Agentic loop: Claude + tool calling via Anthropic SDK
│   ├── app.py                ← Streamlit chat UI
│   └── create_sample.py      ← One-time script: slices data/raw/ → data/sample/
├── mcp/
│   └── server.py             ← MCP server (Phase 6)
└── docs/
    └── decisions.md          ← Running log of design decisions
```

13. Design Decisions Log

Decision	Rationale	Phase
SQLite over PostgreSQL	Local, zero-setup, sufficient for static dataset, portfolio-friendly	1
Streamlit for UI	Python-native, minimal frontend knowledge, fast to build	1
Commodity basket as primary aggregation	6 baskets = clean top-level grouping; matches thesis methodology	1
loss_percentage not loss quantity	Research is fraction-based; consistent with thesis	1
Emissions in kt	Consistent unit across all emissions tables	1
CO2eq IPCC AR5 100-year GWP	Standard methodology; consistent with thesis and FAO reporting	1
Star schema for SQLite	Mirrors thesis data model; clean join structure	1
Full data gitignored, sample data committed	Source data is public (FAO, WDI, PIK, EDGAR), meticulously cleaned for thesis. Full files kept local. Curated sample in data/sample/ committed to power live demo	1
Repo name: food-waste-analytics-chatbot	Clean, descriptive, portfolio-friendly. Lowercase with hyphens is standard GitHub convention	1
4 fact tables not 3	EDGAR (shares 0–1) and PIK (absolute kt) are different value types — must stay separate	2
dev_country column retained	Additional reporting attribute for future analysis	2
Year trim on PIK to 1966+	Analysis scope starts 1966; pre-1966 adds size with no analytical value	2
source_name on all fact tables	Consistent provenance tracking across all tables	2
FAO TIER 1 only for food system emissions	UNFCCC rows are sparse; FAO TIER 1 is complete and consistent	2
Element filter: CH4, N2O, CO2eq AR5 only	Avoids double-counting from per-gas CO2eq conversions	2
KYOTOGHG excluded from PIK	Aggregate metric — individual gases give more value, avoid double-counting	2
GDP 2023 column dropped	Outside analysis scope; WDI data used to 2022 consistently	2
Q1.3 reframed as contextual enrichment	Population/GDP alone don't tell a food waste story	3
Q2.1 broken down by region	Regional breakdown reveals where supply chain losses are most severe	3
Q3.3 added: Crops vs Livestock	Livestock emissions are generally high — important dimension missed in initial scope	3
item_filter as named presets	Filter logic in Python not Claude's reasoning — safer, fewer tokens	4

Decision	Rationale	Phase
group_by as dynamic list	Composable tools serve multiple questions; Claude selects from constrained set	4
limit parameter on all tools	Prevents unbounded result sets consuming context window tokens	4
Q4.1 as single combined tool	Simpler for Claude, cleaner answers — internal join more reliable	4
5 tools not 10	Composable design serves all 10 questions without redundancy	4
DATA_MODE env variable	Controls full vs sample data without code changes	5
Commit → clear → next session rhythm	Preserves continuity while managing tokens — code lives in GitHub not memory	5
FAO emissions m49_code apostrophe prefix	Source file stores m49_code as text with leading apostrophe (e.g. '004). Stripped and cast to int in load_data.py to match dim_region join key	5
WDI GDP data is total GDP not per capita	world_bank_gdp_data.csv contains total GDP in current USD. query_population_gdp returns both avg_total_gdp_usd (as stored) and avg_gdp_per_capita (derived: SUM(gdp)/SUM(population))	5
query_total_ghg_with_food_share uses CTEs	PIK has ~20 rows/country/year, EDGAR has ~9. Direct join inflates SUM by 9×. CTEs pre-aggregate each source before joining to avoid fan-out	5
Streamlit UI: conversation starter cards	4 clickable question cards in 2×2 grid shown when chat is empty (one per analytical group). Cards disappear once conversation starts — clean entry point for portfolio visitors	5
Sidebar: data mode + sources + About	Data mode indicator, source descriptions, and About blurb useful for portfolio visitors and demo context	5
Claude responses rendered as markdown	Claude returns structured markdown (bold, bullets). Rendering improves readability with no extra effort	5
sys.path manipulation in app.py	Streamlit Cloud runs app.py from a different working directory than local. sys.path.insert(0, repo root) ensures 'src' is importable in both environments	5
Sample data: 4 regions, 2005–2022	Europe, SSA, S&SE Asia, LA — covers high/low income spread and all 4 analytical groups. 2005+ keeps files small while preserving trend depth	6
create_sample.py mirrors raw file format	Output files match data/raw/ structure exactly so load_data.py works unchanged in both DATA_MODE=full and DATA_MODE=sample	6
DATA_MODE=sample set as Streamlit secret	Streamlit Community Cloud has no access to raw files. DATA_MODE=sample in Streamlit secrets ensures app loads from committed data/sample/ at runtime	6

14. Glossary

Term	Definition
GHG	Greenhouse gases — CO2, CH4, N2O, F-gases
CO2eq	CO2 equivalent — common GWP metric. 100-year IPCC AR5 values used throughout
IPCC	Intergovernmental Panel on Climate Change
FAO	Food and Agriculture Organization of the United Nations
WDI	World Development Indicators (World Bank)
PIK	Potsdam Institute for Climate Impact Research — source of total economy-wide emissions
EDGAR	Emissions Database for Global Atmospheric Research — source of food emission shares
CPC	Central Product Classification — UN product coding system
M49 code	UN standard country/area code used by FAO data
Agrifood systems	All activities for primary production of food and non-food agricultural products
Agentic loop	Cycle where Claude reads a message, decides whether to call a tool or answer directly, executes tools if needed, returns final response
Tool calling	Anthropic API feature — Claude invokes developer-defined functions to fetch data or perform actions
MCP	Model Context Protocol — standard for exposing tools to Claude. Used in Claude Desktop, VS Code extensions, and workplace integrations
DATA_MODE	Environment variable controlling data source: 'full' loads data/raw/, 'sample' loads data/sample/
NAOCE	North America & Oceania
NAWACA	North Africa, Western & Central Asia
SSEA	South & Southeast Asia
SSA	Sub-Saharan Africa
LA	Latin America

Living document — update phase status and decisions log as the project progresses. | v0.5 — Last updated: Phase 5 complete, Phase 6 (MCP Server) in progress.